

10 — Riunire due rami che hanno costruito la stessa cosa

La domanda a cui risponde: cosa succede quando due linee di lavoro partono dallo stesso commit e costruiscono, senza saperlo, la stessa funzionalità due volte? E perché il conflitto peggiore non è quello che Git ti segnala?

Il fatto

Il 26 luglio 2026 il repository si trovava così: `main` locale con 6 commit, `origin/main` con 19, base comune `e9eedd4`. Nessuno dei due era sbagliato. Semplicemente si era lavorato in locale senza aggiornare prima.

```
git status
# Il tuo branch e 'origin/main' sono diventati divergenti
# e hanno rispettivamente 6 e 19 commit differenti.
```

La prima cosa da capire è che **niente era perso**. Un commit fatto è un oggetto immutabile nel database di Git: finché esiste un riferimento che lo raggiunge, resta. Il rischio non era la perdita, era la *risoluzione frettolosa* — un `git checkout --theirs` distratto su un file, e sei giorni di lavoro sparivano dall'albero pur restando nel repository.

Per questo il primo comando è stato un'etichetta, non un merge:

```
git branch salvataggio-locale-26-07 main
```

Un branch è solo un puntatore a un commit: 41 byte su disco. Costa nulla e rende impossibile il disastro.

Differenza rispetto a Java/Angular

In un progetto Maven o npm il conflitto tipico è su `pom.xml` o `package-lock.json`: fastidioso ma meccanico, due dipendenze da unire. Qui non c'era nessun file di build da unire — e proprio per questo tutti i conflitti erano **semantici**. Git segnalava 23 blocchi su 11 file; solo 4 di quei blocchi erano il problema vero.

Lezione 1 — Il conflitto che Git *non* segnala

Git confronta testo. Se due rami scrivono cose diverse nelle stesse righe, lo dice. Ma se due rami scrivono **due file con lo stesso nome** e contenuto completamente diverso, Git lo classifica come banale «aggiunta/aggiunta» — e se due rami costruiscono la stessa funzionalità in **file con nomi diversi**, Git non dice proprio nulla e il merge riesce pulito.

Era successo entrambe le cose:

Ramo	File	Cosa fa
locale	<code>src/engine/forza.js</code>	<code>avvicinaAForza()</code> : scambia carte per avvicinarsi a un obiettivo
origin	<code>src/engine/forza.js</code>	<code>forza()</code> : misura un mazzo sulla scala 0-100
locale	<code>src/data/riferimento.js</code>	un mazzo salvato eletto a metro di paragone
origin	<code>src/data/mazzi-prefatti.js</code>	i Kit Allenatore misurati, stesso scopo

Il primo caso Git lo segnala. Il secondo no: sono due file nuovi con nomi diversi, il merge sarebbe passato senza una parola, e il progetto si sarebbe ritrovato con due modi paralleli di rispondere alla stessa domanda.

La morale: dopo un merge fra rami molto divergenti, l'elenco dei file toccati dai due lati va letto con gli occhi, cercando *funzionalità* sovrapposte. `git diff --name-only <base> <ramo>` sui due lati è il punto di partenza, non l'arrivo.

Qui la risoluzione è stata tenere entrambe le strategie **in cascata** — `cercaPiano()` sceglie il seme migliore, poi `avvicinaAForza()` rifinisce con gli scambi — e il modulo locale è stato rinominato `src/engine/obiettivo-forza.js`, perché un file per responsabilità è una regola del progetto e «forza» erano due responsabilità diverse che avevano litigato per un nome.

Lezione 2 — Due rami, la stessa versione di schema

`src/data/deposito.js` gestisce le migrazioni di IndexedDB a cascata:

```
if (daVersione < 1) { /* collezione */ }
if (daVersione < 2) { /* mazzi */ }
if (daVersione < 3) { /* ... */ }
```

Il ramo locale aveva aggiunto lo store `impostazioni` come **versione 3**. Il ramo remoto aveva aggiunto lo store `prezzi` come **versione 3**. Entrambi corretti, isolatamente.

Il problema è che uno schema di database non vive solo nel repository: vive **sui dispositivi**. Un telefono che aveva aperto la versione pubblicata aveva già `pokedex` a v3 con `prezzi` dentro. Il browser di sviluppo aveva v3 con `impostazioni`. Due database diversi che dichiarano lo stesso numero.

Unire ingenuamente — «metto tutti e due sotto `daVersione < 3`» — significa che sul dispositivo già a v3 il blocco non gira mai, e lo store mancante non viene creato: l'app si rompe alla prima lettura. Alzare a 4 e basta significa che sul browser che ha già `impostazioni` la `createObjectStore()` solleva `ConstraintError` — e questo è il dettaglio cattivo:

Un'eccezione dentro `onupgradeneeded` **annulla l'intera transazione di aggiornamento**, compresi i passi già andati a buon fine.

La soluzione è creare in modo idempotente:

```
function creaSeManca(db, nome) {
  if (!db.objectStoreNames.contains(nome)) {
    db.createObjectStore(nome, { keyPath: 'id' });
  }
}
```

Chi viene da Java riconosce il pattern: è `CREATE TABLE IF NOT EXISTS`, ed è esattamente il motivo per cui Flyway e Liquibase numerano le migrazioni con timestamp invece che con interi progressivi. Un intero è un nome che due persone possono scegliere insieme senza accorgersene.

Lezione 3 — Due scale numeriche che si somigliano

Il difetto più insidioso non era in nessun conflitto. Il ramo locale calcolava la forza del mazzo di riferimento così:

```
forza: piano.equilibrio?.punteggi?.[indice]?.totale ?? null,
```

Quel numero viene da `bilancia.js`: è una scala **relativa**, serve a confrontare i mazzi di uno stesso piano fra loro, e nei test valeva 132 e 125. Il ramo remoto aveva introdotto `forza.js`, scala **assoluta 0-100**.

Dopo il merge, quel valore da 132 finiva a `bersaglioPer()` come bersaglio su una scala che arriva a 100. Due numeri, entrambi chiamati «forza», entrambi plausibili a occhio — e nessun test rosso, perché i test del ramo locale verificavano coerentemente la scala vecchia:

```
assert.deepEqual(descriviMazzo(piano, 1), { ... forza: 125, ... });
```

Un test verde non prova che il codice sia giusto: prova che è coerente con ciò che il test si aspetta. Quando due rami si uniscono, le aspettative di uno possono essere diventate sbagliate, e il test le protegge invece di segnalarle. Qui è stato riscritto per fissare la regola vera:

```
test('la forza non viene dai punteggi di equilibrio', () => {
  const descrizione = descriviMazzo(piano, 1);
  assert.notEqual(descrizione.forza, 125);
});
```

```
assert.ok(descrizione.forza <= 100);
});
```

Lezione 4 — Il guardiano che ha funzionato

Non tutto è andato scoperto a mano. `tests/guscio.test.js` verifica che l'elenco GUSCIO di `sw.js` e i file sotto `src/` coincidano **nei due sensi**, e alla prima esecuzione dopo il merge ha detto:

```
ogni modulo del progetto è precaricato
+ [ './src/engine/obiettivo-forza.js' ]
```

Il merge aveva anche prodotto una riga duplicata (`./src/engine/forza.js` elencato due volte, una per ramo) — l'impronta esatta di due rami che aggiungono la stessa riga in punti diversi della stessa lista.

Questo è il valore di un test che confronta due elenchi che *devono* dire la stessa cosa. È lo stesso principio dell'`architecture test` con ArchUnit in Java: non prova il comportamento, prova un **invariante strutturale** che nessuno si ricorda di controllare a mano.

Il procedimento, in ordine

1. **Etichetta prima di toccare:** `git branch salvataggio-<data>`.
2. **Misura la divergenza:** `git merge-base`, poi `git diff --name-only` sui due lati. Cerca sovrapposizioni di *funzionalità*, non di righe.
3. **Fermati e chiedi** quando due rami hanno costruito la stessa cosa: è una decisione di prodotto, non di merge.
4. **Risolvi prima il meccanico** (versioni, elenchi, import): libera la testa per il resto.
5. **Sospetta dei test verdi** dei file che hai unito.
6. **Verifica nel browser**, non solo con `node --test`: il cablaggio del DOM non è coperto da nessuna suite.

Sul punto 6, un dettaglio che costa mezz'ora se non lo si sa: durante la verifica il **service worker serviva ancora i file vecchi dalla cache**, e le modifiche appena scritte sembravano non avere effetto. È lo stesso meccanismo descritto in 01 — Progressive Web App, visto dal lato fastidioso. In sviluppo:

```
for (const r of await navigator.serviceWorker.getRegistrations()) await r.unregister();
for (const k of await caches.keys()) await caches.delete(k);
```

Domande di verifica

1. Perché `git branch` prima di un merge difficile costa praticamente nulla? Cosa contiene davvero un branch?
2. Due rami aggiungono uno store IndexedDB numerandolo entrambi versione 3. Perché non basta alzare `VERSIONE_DB` a 4 e mettere le due `createObjectStore()` sotto `daVersione < 4`?
3. Nel merge sono stati uniti `cercaPiano()` (prova più semi) e `avvicinaAForza()` (scambia carte). Perché l'ordine conta, e cosa succederebbe invertendolo?
4. `tests/guscio.test.js` verifica la corrispondenza **nei due sensi**. Quale dei due versi ha causato in passato il blocco silenzioso degli aggiornamenti descritto nel `CLAUDE.md`, e perché quel verso è il più pericoloso?